Practical - 9

Aim: Given two strings str1 and str2 and below operations that can be performed on str1. Find minimum number of edits (operations) required to convert 'str1' into 'str2'.
Insert Remove Replace, All of the above operations are of equal cost.

PROGRAM CODE :

```
def edit_distance(str1, str2):
    m = len(str1)
    n = len(str2)

    # Create a DP table to store results of subproblems
    dp = [[0 for x in range(n + 1)] for x in range(m + 1)]

    # Fill dp[][] bottom up
    for i in range(m + 1):
        for j in range(n + 1):

            # If str1 is empty, we need to insert all characters of str2
            if i == 0:
                dp[i][j] = j    # j insertions

            # If str2 is empty, we need to remove all characters of str1
            elif j == 0:
                dp[i][j] = i    # i removals

            # If last characters are the same, no cost, move diagonally
            elif str1[i - 1] == str2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
```

```
            # If last characters are different, consider all possibilities
            # and take the minimum: insert, remove, or replace
            else:
                dp[i][j] = 1 + min(dp[i][j - 1],      # Insert
                                   dp[i - 1][j],      # Remove
                                   dp[i - 1][j - 1])  # Replace

    # The answer will be in dp[m][n]
    return dp[m][n]

# Take input from the user
str1 = input("Enter the first string (str1): ")
str2 = input("Enter the second string (str2): ")

# Compute and display the result
result = edit_distance(str1, str2)
print(f"Minimum edits required to convert '{str1}' into '{str2}' are: {result}")
```